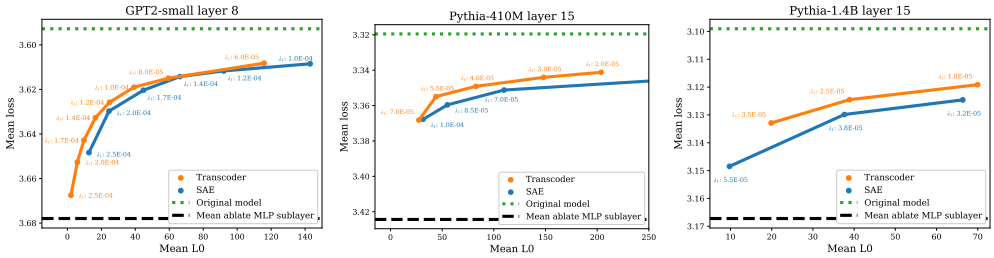

Figure 3: The sparsity-accuracy tradeoff of **transcoders** versus **SAEs** on GPT2-small, Pythia-410M, and Pythia-1.4B. Each point corresponds to a trained SAE or transcoder, and is labeled with the L1 regularization penalty λ_1 used during training.

trained on MLP-out activations (as these are the activations that MLP SAEs are typically trained on). Due to compute limitations, we used the same learning rate, which was determined via a hyperparameter sweep on *transcoders*, for both SAEs and transcoders. This means that the learning rate might not be optimal for SAEs. Nevertheless, we did perform a separate hyperparameter sweep of λ_1 for the SAEs and transcoders.

We evaluated each SAE and transcoder on the same 3.2M tokens of OpenWebText data [21]. We also recorded the loss of the unmodified and mean-ablated model (always replacing the MLP sublayer output with its mean output over the dataset) as best- and worst-case bounds, respectively.

We summarize the Pareto frontiers of the sparsity-accuracy tradeoff for all models in Figure 3. In all cases, transcoders are equal to or better than SAEs. In fact, the gap between transcoders and SAEs seems to widen on larger models. Note, however, that compute limitations prevented us from performing more exhaustive hyperparameter sweeps; as such, it might be possible that a different set of hyperparameters could have allowed SAEs to surpass transcoders. Nonetheless, these results make us optimistic that using transcoders incurs no penalties versus SAEs trained on MLP activations.

5 Circuit analysis case studies

5.1 Blind case study: reverse-engineering a feature

To understand the utility of transcoders for circuit analysis, we carried out nine **blind case studies**, where we randomly selected individual transcoder features in a ninth-layer (of 12) GPT2-small transcoder and used circuit analysis to form a hypothesis about the semantics of the feature—*without looking at the text of examples that cause the feature to activate*. In blind case studies, we use a combination of input-invariant and input-dependent information to allow us to evaluate transcoders as a tool to infer model behavior with minimal prompt information. This better reflects a key goal of mechanistic interpretability: to be able to understand model behavior on unknown, unforeseen tasks.

In contrast, reverse-engineering a feature where one already has an idea of its behavior can introduce confirmation bias. For instance, looking at activation patterns prior to circuit analysis can predispose a researcher to seek out only circuits that corroborate their interpretation of these activation patterns, potentially ignoring circuits that reveal other information about the feature. Conversely, if the circuit analysis method is faulty and yields some explanations that are not reflected in the feature activations, then the researcher might ignore those spurious explanations and thus obtain an overly-positive assessment of the circuit analysis method. The "rules of the game" for blind case studies are that:

1. The specific tokens contained in any prompt are not allowed to be directly seen. As such, prompts and tokens can only be referenced by their index in the dataset.
2. These prompts may be used to compute input-dependent information (activations and circuits), as long as the tokens themselves remain hidden.
3. Any input-invariant information, including feature de-embeddings, is allowed.

In this section, we summarise a specific blind case study, how we used our circuits to reverse-engineer feature 355 in our layer 8 transcoder. Other studies, as well as a longer description of the study summarized here, can be found in App. H.

Note that we use the following compact notation for transcoder features: `tcA[B]@C` refers to feature `B` in the layer `A` transcoder at token `C`.

Building the first circuit. We started by getting a list of indices of the top-activating prompts in the dataset for `tc8[355]`. Importantly, we did not look at the actual tokens in these prompts, as doing so would violate Rule 1. For our first input, we chose example 5701, token 37; `tc8[355]` fires at strength 11.91 on this token in this input. Our greedy algorithm for finding the most important computational paths for causing `tc8[355]@37` to fire revealed contributions from the current token (37) and earlier tokens (like 35, 36, and 31).

Current-token features. From token 37, we found strong contributions from `tc0[16632]@37` and `tc0[9188]@37`. Input-invariant de-embeddings of these layer 0 features revealed that they primarily activate on variants of `;`, suggesting that token 37 contributed to the feature by virtue of being a semicolon. Another feature which contributed strongly through the current token, `tc6[11831]`, showed a similar pattern. Among the top *input-invariant* connections from layer 0 transcoder features to `tc6[11831]`, we once again found the same semicolon features `tc0[16632]` and `tc0[9188]`.

Previous-token features. Next we checked computational paths from previous tokens through attention heads. Looking at these contextual computational paths revealed a contribution from `tc0[13196]@36`; the top de-embeddings for this feature were years like `1973`, `1971`, `1967`, and `1966`. Additionally, there was a contribution from `tc0[10109]@31`, for which the top de-embedding was `(`.

Furthermore, there was a contribution from `tc6[21046]@35`. The top input-invariant connections to this feature from layer 0 were `tc0[16382]` and `tc0[5468]`. The top de-embeddings for the former were tokens associated with Eastern European last names (e.g. `kowski`, `chenko`, `owicz`) and the top de-embeddings for the latter feature were English surnames (e.g. `Burnett`, `Hawkins`, `Johnston`). This heavily suggested that `tc6[21046]` was a surname feature.

Thus, the circuit revealed this pattern was important to our feature: "`(`-[?]-[?]-[?]-[surname]-[year]-`;`".

Analysis. We hypothesized that `tc8[355]` fires on semicolons in parenthetical citations like "(Vaswani et al. 2017**;** Elhage et al. 2021)". Further investigation on another input yielded a similar pattern—along with a feature whose top de-embedding tokens included `Accessed`, `Retrieved`, `Neuroscience`, and `Springer`. This bolstered our hypothesis even more.

Here, we decided to end the blind case study and check if our hypothesis was correct. Sure enough, the top activating examples included semicolons in citations such as "(Poeck, 1969; Rinn, 1984)" and "(Robinson et al., 1984; Starkstein et al., 1988)". We note that the first of these is the example at index $(5701, 37)$ we analyzed above.

"Restricted" blind case studies. Because MLP0 features tend to be single-token, significant information about the original prompt can be obtained by looking at which MLP0 transcoder features are active and then taking their de-embeddings. In order to address this and more fully investigate the power of input-invariant circuit analysis, six of the eight case studies that we carried out were **restricted blind case studies**, in which all input-dependent MLP0 feature information is forbidden to use. For more details on these case studies, see Appendix H.2.

5.2 Analyzing the GPT2-small "greater-than" circuit

We now turn to address the "greater-than" circuit in GPT2-small previously considered by Hanna et al. [25]. They considered the following question: given a prompt such as "The war lasted from 1737 to 17", how does the model know that the predicted next year token has to be greater than 1737? In their original work, they analyzed the circuit responsible for this behavior and demonstrated that MLP10 plays an important role, looking into the operation of MLP10 at a neuronal level. We now apply transcoders and the circuit analysis tools accompanying them to this same problem.

5.2.1 Initial investigation

First, we used the methods from Sec. 3.2.3 to investigate a single prompt and obtain the computational paths most relevant to the task. This placed a high attribution on MLP10 features, which were in turn activated by earlier-layer features mediated by attention head 1 in layer 9. This corroborates the analysis in the original work.

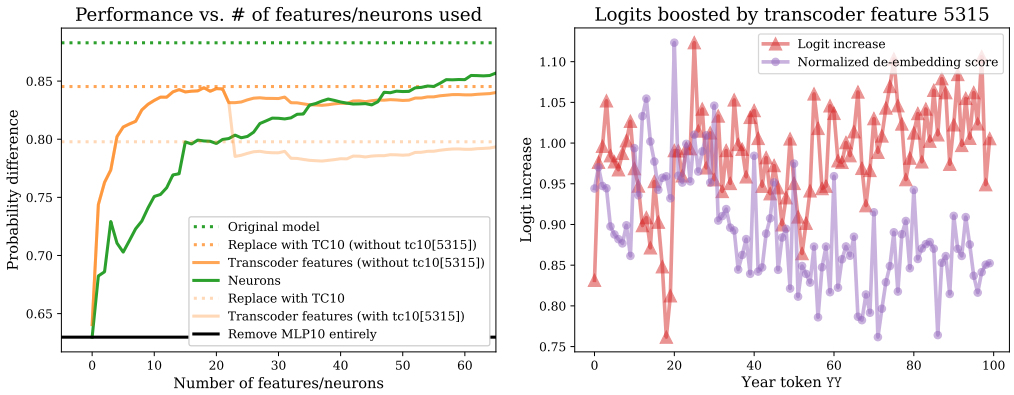

Figure 4: **Left:** Performance according to the probability difference metric when all but the top k **features** or **neurons** in MLP10 are zero-ablated. **Right:** The **DLA** and **de-embedding score** for `tc10[5315]`, which contributed negatively to the transcoder's performance.

Next, we investigated which MLP10 transcoder features were most important on a variety of prompts, and how their activations are mediated by attention head 1 in layer 9. Following the original work, we generated all 100 prompts of the form "The war lasted from 17YY to 17", where YY denotes a two-digit number. We found that the MLP10 features with the highest variance in activations over this set of prompts also had top input-dependent connections from MLP0 features through attention head 1 in layer 9 whose top de-embeddings were two-digit numbers. The top *input-invariant* connections from MLP0 features through attention head 1 in layer 9 to MLP10 features *also had two-digit numbers among their top de-embedding tokens*. This positive result was somewhat unexpected, given that there are only 100 two-digit number tokens in the model's vocabulary of over 50k tokens.

We then used **direct logit attribution (DLA)** [15] to look at the effect of each transcoder feature on the predicted logits of each YY token in the model's vocabulary. These results, along with de-embedding scores for each YY token, can be seen in Figure 5. The de-embeddings scores are highest for YY tokens where years following them are boosted and years preceding them are inhibited.

5.2.2 Comparison with neuronal approach

Next, we compared the transcoder approach to the neuronal approach to see whether transcoders give a *sparser* description of the circuit than MLP neurons do. To do this, we computed the 100 highest-variance layer 10 transcoder features and MLP10 neurons. Then, for $1 \leq k \leq 100$, we zero-ablated all but the top k features in the transcoder/neurons in MLP10 and measured how this affected the model's performance according to the **mean probability difference** metric presented in the original paper. We also evaluated the original model with respect to this metric, along with the model when MLP10 is replaced with the full transcoder.

The results are shown in the left half of Figure 4. For fewer than 24 features, the transcoder approach outperforms the neuronal approach; its performance drops sharply, however, around this point. Further investigation revealed that `tc10[5315]`, the 24th-highest-variance transcoder feature, was responsible for this drop in performance. The DLA for this feature is plotted in the right half of Figure 4. Notice how, in contrast with the three highest-variance transcoder features, `tc10[5315]` displays a flatter DLA, boosting all tokens equally. This might explain why it contributes to poor performance. To account for this, note that the left half of Figure 4 also demonstrates the performance of the transcoder when this "bad feature" is removed.

While the transcoder does not recover the full performance of the original model, it needs only a handful of features to recover most of the original model's performance; many more MLP neurons are needed to achieve the same level of performance. This suggests that the transcoder is particularly useful for obtaining a sparse, understandable approximation of MLP10. Furthermore, the transcoder features suggest a simple way that the MLP10 computation may (approximately) happen: by a small set of features that fire on years in certain ranges and boost the logits for the following years.